

Spring MVC and Hibernate ORM

Notes

Goal of the chapter

This chapter will discuss about Hibernate ORM framework and how this is used to save persistent objects into the database.

1 Hibernate object-relational mapping (ORM) Java framework

Hibernate is an object-relational mapping Java framework that simplifies the development of Java application to interact with the database. It is an open source framework which provides APIs to map the java classes to relational database tables and Java data types to SQL data types. A Hibernate ORM architecture diagram has been presented in the figure below [1](#)

Different objects are created to retrieve data from DBMS and save data to the DBMS using persistent class and Hibernate ORM objects. Configuration object provides the configuration file used by the Hibernate ORM. A sample configuration file has been presented below from the the Webapp of Task 5.2C.

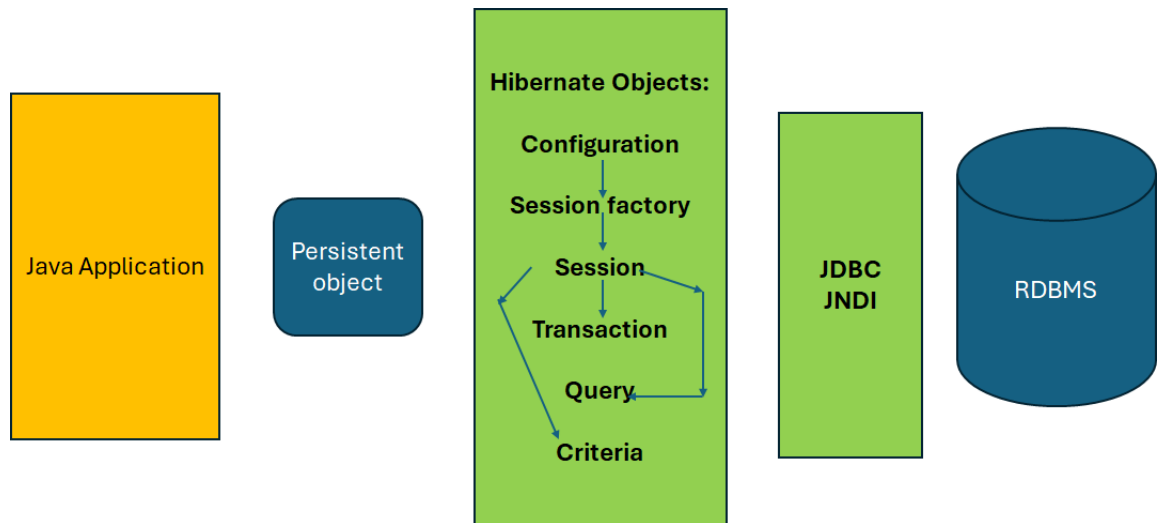


Figure 1: Hibernate ORM architecture

```

1  <!DOCTYPE hibernate-configuration PUBLIC
2      "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
3      "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd"
4  >
5  <hibernate-configuration>
6
7      <session-factory>
8
9          <!-- JDBC Database connection settings -->
10         <property
11             name="connection.driver_class">com.mysql.cj.jdbc.Driver</
12         property>
13         <property
14             name="connection.url">jdbc:mysql://localhost:3306/sessiondb</
15         property>
16         <property name="connection.username">coachdbadmin</property>
17         <property name="connection.password">coachdbadmin</property>
18
19         <!-- JDBC connection pool settings ... using built-in test
20         pool -->
21         <property name="connection.pool_size">1</property>
22
23         <!-- Select our SQL dialect -->
24         <property name="dialect">org.hibernate.dialect.MySQLDialect</
25         property>
26
27         <!-- Echo the SQL to stdout -->
28         <property name="show_sql">true</property>
29         <property name="format_sql">true</property>
30         <property name="use_sql_comments">true</property>
31
32         <!-- Set the current session context -->
33         <property name="current_session_context_class">thread</property>
34     </session-factory>
35 </hibernate-configuration>
  
```

As you can see in the above configuration file, it shows the database connection URL, username and password to access the RDBMS in the property tag. The hierarchy of object creation has been presented in Figure 1. A SessionFactory object is created from the Configuration object of above which works as as a factory for session objects. A session object is created from the SessionFactory object. A session object will keep track of the changes that is made to java persistent objects, and make sure that they are mapped or sync with the database. A transaction object is created from the Session object which represents an atomic unit of work done on the persistent object.

1.1 Persistent object

Persistent object is created from persistent class which are saved in the databases. A persistence class should have a default constructor (no argument). All attributes should be declared private. `getAttributeName` and `setAttributeName` methods should be defined to access the class attributes with public access specifier. An example of persistent class has been presented below from your Task 5.2C webapp

```

1 package edu.deakin.sit218.coachwebapp.entity;
2
3 @Entity
4 @Table(name = "user")
5 public class Client {
6     @Id
7     @GeneratedValue(strategy=GenerationType.IDENTITY)
8     @Column(name = "iduser")
9     protected int id;
10    public int getId() {
11        return id;
12    }
13    public void setId(int id) {
14        this.id = id;
15    }
16    @Column(name = "username")
17    protected String username;
18
19    @Column(name = "password")
20    protected String password;
21
22    @Column(name = "age")
23    protected int age;
24
25    @Column(name = "workouts")
26    protected int workouts = 0;
27
28    public Client() {
29    }
30
31    public String getUsername() {
32        return username;
33    }
34
35    public void setUsername(String username) {
36        this.username = username;
37    }
38
39    public String getPassword() {
40        return password;
41    }
42
43    public void setPassword(String password) {
44        this.password = password;
45    }
46
47    public int getAge() {
48        return age;
49    }
50
51    public void setAge(int age) {
52        this.age = age;
53    }
54
55    public int getWorkouts() {
56        return workouts;
57    }
58
59    public void setWorkouts(int workouts) {
60        this.workouts = workouts;
61    }
62
63    @Override
64    public String toString() {
65        return "client = [name: "+getUsername()+", age: "+getAge()+
66            ", workouts: "+ getWorkouts()+"]";
67    }
68
69    public void addWorkout() {
70        workouts++;
71    }
72
73 }
74
75 }
76

```

1.2 States of the persistent objects in Hibernate

The states of a persistent object in Hibernate can be any of the following three states: Transient, Persistent and Detached. if the object is not associated with the hibernate session, then it is in the transient state. if the object is associated with the hibernate session, then it is in the persistent state. When the hibernate session is closed, then the object is in the detached state. The following piece of code is from your webapp in Task 5.2C (ClientDAOImpl.java).

In the code below, a SessionFactory object "factory" has been created from a configuration object using new Configuration() and related configuration file in the method public ClientDAOImpl(). A session object is created using: Session session = factory.getCurrentSession() in the public void insertClient(Client client).

Here getCurrentSession() creates and returns a new session if does not exist or returns an existing one if one already exists.

Then session.save(client) in public void insertClient(Client client) persists the client object from a transient state or client object is in now a persistence state from a transient state.

session.close() in finally block closes the hibernate session and the client object is detached. session.beginTransaction() in public void insertClient(Client client) start hibernate transaction as mentioned in the RDBMS configuration parameters (hibernate.cfg.xml) file.

session.getTransaction().commit() in public void insertClient(Client client) commits the transaction.

session.update(client) in public void updateClient(Client client) will update client object in the database, it is used when object has an identifier.

```

1 package edu.deakin.sit218.coachwebapp.dao;
2
3 import java.util.List;
4
5 import org.hibernate.Session;
6 import org.hibernate.SessionFactory;
7 import org.hibernate.cfg.Configuration;
8 import org.hibernate.query.Query;
9
10 import edu.deakin.sit218.coachwebapp.entity.Client;
11
12
13 public class ClientDAOImpl implements ClientDAO{
14
15     SessionFactory factory;
16
17     public ClientDAOImpl() {
18         factory = new Configuration()
19             .configure("hibernate.cfg.xml")
20             .addAnnotatedClass(Client.class)
21             .buildSessionFactory();
22     }
23
24
25     public void updateClient(Client client) {
26         Session session = factory.getCurrentSession();
27         try {
28             session.beginTransaction();
29             session.update(client);
30             session.getTransaction().commit();
31         }
32         finally {
33             session.close();
34         }
35     }
36
37     public void insertClient(Client client) {
38         Session session = factory.getCurrentSession();
39         try {
40             session.beginTransaction();
41             session.save(client);
42             session.getTransaction().commit();
43         }
44         finally {
45             session.close();
46         }
47     }
48 }
49
50
51

```

Code-2: Hibernate ORM and state of the objects

2 Hibernate Query Language (HQL)

HQL is fully object oriented query language. HQL has SELECT, UPDATE, INSERT and DELETE statements similar to SQL

HQL study

You need to study HQL. There are many resources available online.
e.g. https://access.redhat.com/documentation/en-us/red_hat_jboss_enterprise_application_platform/7.2/html/developing_hibernate_applications/hibernate_query_language

```

1 package edu.deakin.sit218.coachwebapp.dao;
2
3 import java.util.List;
4
5 import org.hibernate.Session;
6 import org.hibernate.SessionFactory;
7 import org.hibernate.cfg.Configuration;
8 import org.hibernate.query.Query;
9
10 import edu.deakin.sit218.coachwebapp.entity.Client;
11
12
13 public class ClientDAOImpl implements ClientDAO{
14
15     SessionFactory factory;
16
17     public ClientDAOImpl() {
18         factory = new Configuration()
19             .configure("hibernate.cfg.xml")
20             .addAnnotatedClass(Client.class)
21             .buildSessionFactory();
22     }
23
24     //.....Some more code .....
25
26     public Client retrieveClient(String username) {
27         Session session = factory.getCurrentSession();
28         try {
29             //use the session object to check for a client
30             //start a transaction
31             session.beginTransaction();
32
33             //create the query
34             String hql = "from Client where username = :username";
35             Query<Client> query = session.createQuery(hql);
36             query.setParameter("username", username);
37             List<Client> clients = query.getResultList();
38             if (clients.isEmpty()) {
39                 throw new RuntimeException("Broken precondition: "
40                     + "credentials should be have been verified earlier");
41             }
42             else if (clients.size() > 1) {
43                 throw new RuntimeException("More than one client exists");
44             }
45             else {
46                 return clients.get(0);
47             }
48         }
49         finally {
50             session.close();
51         }
52     }
53
54     public boolean areCredentialsCorrect(String username, String password)
55     {
56         Session session = factory.getCurrentSession();
57         try {
58             session.beginTransaction();
59             //create the query
60             String hql = "from Client where username = :username"+
61                 " and password = :password";
62             Query<Client> query = session.createQuery(hql);
63             query.setParameter("username", username);
64             query.setParameter("password", password);
65             //Obtain the query results
66
67             List<Client> clients = query.getResultList();
68             return !clients.isEmpty();
69         }
70         finally {
71             session.close();
72         }
73     }
74 }
75

```


When you are using Hibernate, you don't need to write java code to create the database connection. Note the difference between database connection creation process with the Hibernate platform and without the hibernate. Following code shows a piece of code without Hibernate platform (Java Standard Edition Service Provider mechanism using jdbc.drivers).

Note in Code-4, `DriverManager.getConnection(jdbcURL)` is used to create connection object.

Also `statement.executeQuery(query)` is used to execute the query. The variable is included as part of SQL statement `String query = "SELECT * FROM client WHERE name = '"+name+"'"`. Here `+name+` is a variable. Variable value is directly concatenated with SQL statements and considered as part of SQL statements. So it is vulnerable to injection attack.

However, Hibernate does not require any connection (Code-3), these have been placed in the Hibernate configuration file. In the public `Client retrieveClient(String username)` (Code-3), named parameter has been placed in the query using a `":"` character, `String hql = "from Client where username = :username"`. The named parameter `:username` is attached with the actual variable `username` using a statement: `query.setParameter("username", username)`. This parameter binding will consider the input as a value which was passed through `public Client retrieveClient(String username)` before.

```

1 package edu.deakin.sit218.coachwebapp.test;
2
3 import java.util.Scanner;
4 import java.sql.Connection;
5 import java.sql.DriverManager;
6 import java.sql.PreparedStatement;
7 import java.sql.ResultSet;
8 import java.sql.SQLException;
9 import java.sql.Statement;
10 public class SQLInjectionTest {
11
12
13     public static void main(String[] args) {
14         //Ask user to give his/her name
15         Scanner scanner = new Scanner(System.in);
16         System.out.print("Enter your name: ");
17         String name = scanner.nextLine();
18         scanner.close();
19
20         String jdbcURL = "jdbc:mysql://localhost/coachappdb?" +
21             "user=coachdbadmin&password=coachdbadmin";
22
23         try {
24             System.out.println("Connecting to Database");
25             Connection conn =
26                 DriverManager.getConnection(jdbcURL);
27
28             //Perform a query to select all users whose name
29             //is equal to the name entered in the command line\
30
31             Statement statement = conn.createStatement();
32             String query = "SELECT * FROM client WHERE name = '"+name+"'";
33             System.out.print("Executing query: ");
34             System.out.println(query);
35             ResultSet result = statement.executeQuery(query);
36
37
38             PreparedStatement statement = conn.prepareStatement("select * from
39                 client where name = ?");
40             statement.setString(1, name);
41             System.out.print(statement.toString() + "\n");
42             ResultSet result = statement.executeQuery();
43
44             //Displaying query results
45             while (result.next()) {
46                 int clientID = result.getInt("id");
47                 int workouts = result.getInt("workouts");
48                 int age = result.getInt("age");
49                 name = result.getString("name");
50                 System.out.println(clientID + ", " + name + ", " + age +
51                     ", " + workouts);
52             }
53             System.out.println("Connection successful!");
54         } catch (SQLException ex) {
55             // handle any errors
56             System.out.println("SQLException: " + ex.getMessage());
57             System.out.println("SQLState: " + ex.getSQLState());
58             System.out.println("VendorError: " + ex.getErrorCode());
59         }
60     }
61 }
62 }
63
64
65

```

Code-4: jdbc.drivers based connection mechanism

3 Execution flow

The MVC diagram for spring hibernate has been shown in Figure 2. The browser's requests is received by the tomcat server. Here Index.jsp page is opened according to the welcome file in web.xml. It goes to the front controller (dispatch) servlet according to the web.xml file. The front controller receives request object and create a model object. Based on the Index.Jsp (response.sendRedirect("showLogin")), front controller finds corresponding controller's request mapping. In this case this is LoginController class (Code-5). It finds @RequestMapping("/showLogin") then, corresponding method handler is executed which is public String login() . LoginController returns the view name to the Dispatch. Dispatch identifies the view location as mentioned in spring configuration file. It calls the view and passes model objects. In this case the view is plain-login.jsp which receives username and password. The response from the view is returned to Dispatch which has a URL requests for *authenticateTheUser* . Now Dispatch finds corresponding controller and request mapping handler for *authenticateTheUser* which is public String authenticateTheUser() method. Dispatch also passes updated model and request objects to the handler.

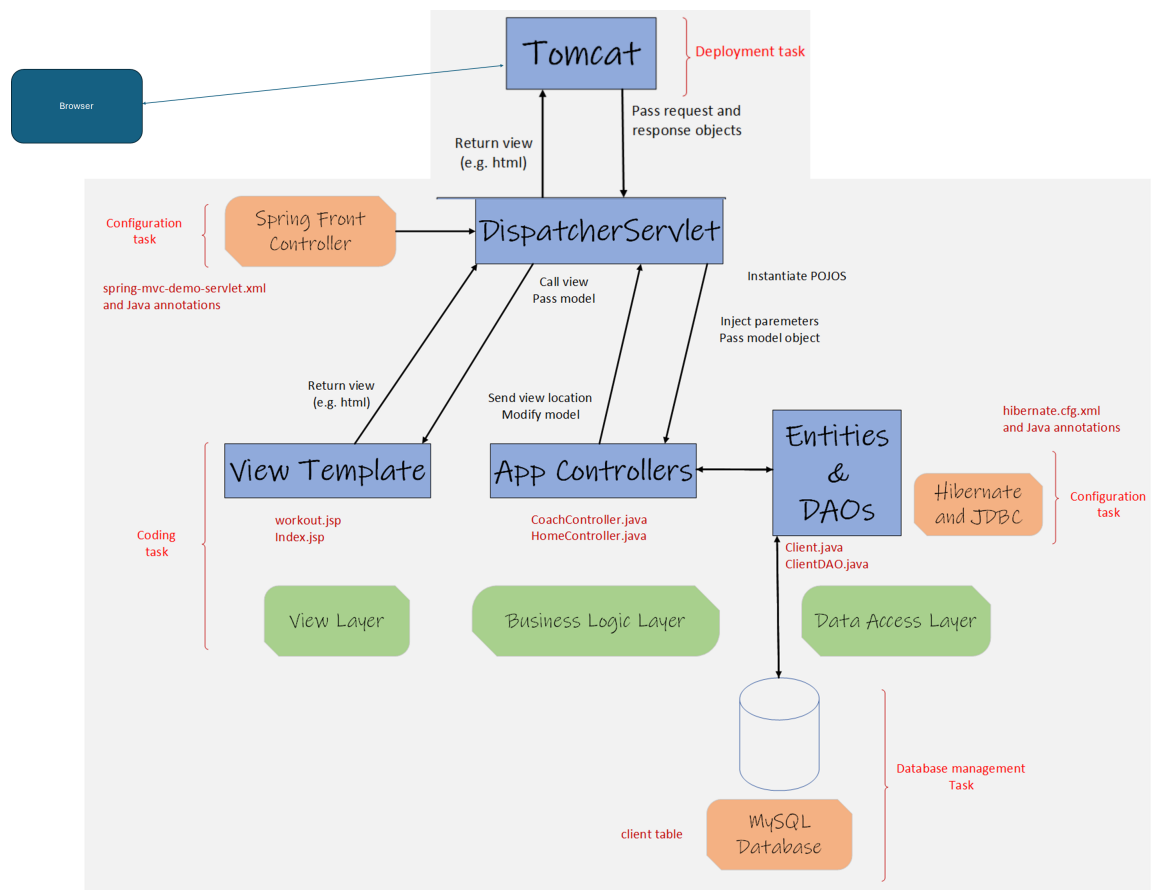


Figure 2: MVC diagram for spring Hibernate architecture

```

1
2 package edu.deakin.sit218.coachwebapp.controller;
3
4
5 import javax.servlet.http.HttpServletRequest;
6 import javax.servlet.http.HttpSession;
7 import javax.validation.Valid;
8
9 import org.springframework.stereotype.Controller;
10 import org.springframework.ui.Model;
11 import org.springframework.validation.BindingResult;
12 import org.springframework.web.bind.annotation.ModelAttribute;
13 import org.springframework.web.bind.annotation.RequestMapping;
14 import org.springframework.web.bind.annotation.RequestParam;
15 import org.springframework.web.bind.annotation.RequestMethod;
16
17 import edu.deakin.sit218.coachwebapp.dao.ClientDAO;
18 import edu.deakin.sit218.coachwebapp.dao.ClientDAOImpl;
19 import edu.deakin.sit218.coachwebapp.entity.Client;
20
21 @Controller
22 public class LoginController {
23
24
25
26     @RequestMapping("/showLogin")
27     public String login() {
28         return "plain-login";
29     }
30
31     @RequestMapping("/authenticateTheUser")
32     public String authenticateTheUser(
33         @RequestParam("username") String username,
34         @RequestParam("password") String password,
35         Model model, HttpServletRequest request) {
36
37         ClientDAO dao = new ClientDAOImpl();
38
39         if (dao.areCredentialsCorrect(username, password)) {
40             //Logic for when the client is exists
41             Client client = dao.retrieveClient(username);
42             //Now that the client is authenticated
43             //We handle the logic to the CoachController.
44             /*
45             HttpSession session = request.getSession();
46             session.setAttribute("user", client.getUsername());
47             return "redirect:/workout";
48             */
49             return "redirect:/workout?user="+client.getId();
50         }
51         else {
52             //Logic for when the client doesn't exist
53             model.addAttribute("errorMessage", "Wrong combination of password
54             and username");
55             return "plain-login";
56         }
57     }
58 }
59
60 }
61
62
63

```

Code-5: Front controller LoginController.java

References

- <https://docs.jboss.org/hibernate/orm/3.3/reference/en/html/queryhql.html>
- <https://www.w3schools.blog/hibernate-framework>